

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Wyświetlanie liczb na wyświetlaczach 7-segmentowych. Kod BCD

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 4

Studia dzienne

Informatyka

Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 24.03.2022

Zadanie 1

Wyświetl dwucyfrową liczbę $A = a_1a_0$ w kodzie BCD na ekranach HEX1 i HEX0, informuj w przypadku nieprawidłowych kodów numerycznych: 1010 - 1111, z następującym przyporządkowaniem pinów:

Wyprowadzenia	Piny
a1	SW[7:4]
a0	SW[3:0]
error a1	LEDR[9]
error a0	LEDR[8]
[a1]	HEX1
[a0]	HEX0

Wartości przełączników SW [7:0] mają być również wyświetlane na LEDR [7:0].

Wskazówka: opracuj projekt **decoder_hex_10** (input[3:0]x, output[0:6]h).

Kod w Verilog

Moduł najwyższego poziomu: bcd_2_digit.

```
module bcd_2_digit(  
    input [7:0] SW,  
    output [9:0] LEDR,  
    output [6:0] HEX1, HEX0);  
  
    assign LEDR[7:0] = SW[7:0];  
    decoder_hex_10 dec1(SW[7:4], HEX1[6:0]);  
    bcd_validator val1(SW[7:4], LEDR[9]);  
    decoder_hex_10 dec0(SW[3:0], HEX0[6:0]);  
    bcd_validator val0(SW[3:0], LEDR[8]);  
  
endmodule
```

```
module decoder_hex_10(  
    input [3:0] x,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (x)  
            4'd0: h = 7'b1000000; // 64  
            4'd1: h = 7'b1111001; // 121  
            4'd2: h = 7'b0100100; // 36  
            4'd3: h = 7'b0110000; // 48  
            4'd4: h = 7'b0011001; // 25  
            4'd5: h = 7'b0010010; // 18  
            4'd6: h = 7'b0000010; // 2  
            4'd7: h = 7'b1111000; // 120  
            4'd8: h = 7'b0000000; // 0  
            4'd9: h = 7'b0010000; // 16  
            default: h = 7'b1111111; // 127  
        endcase  
  
endmodule
```

```

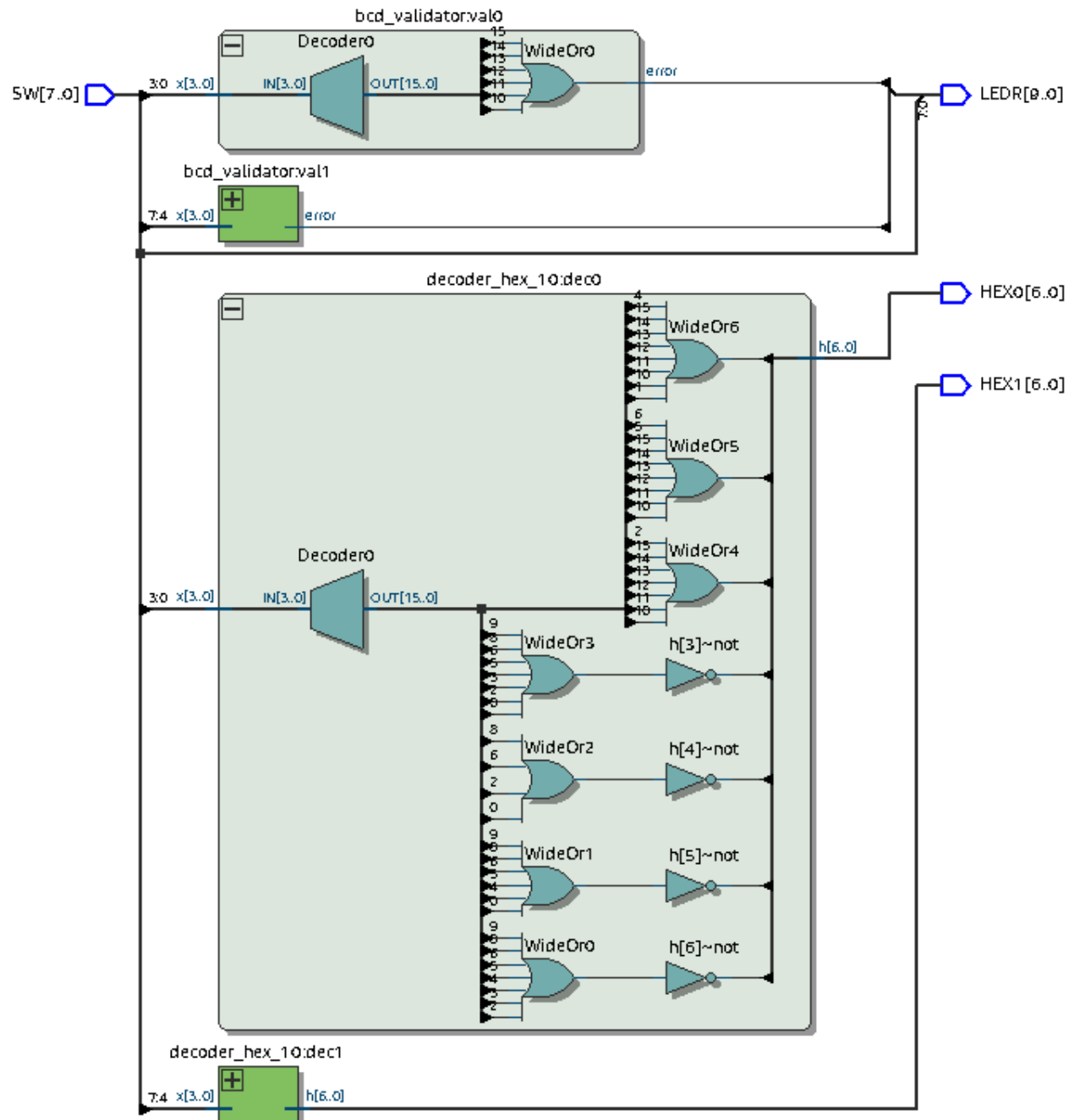
module bcd_validator(
    input [3:0] x,
    output reg error);

    always @ (*)
        case (x)
            4'd10, 4'd11, 4'd12, 4'd13, 4'd14, 4'd15: error = 1'b1;
            default: error = 1'b0;
        endcase

endmodule

```

Wyniki syntezy



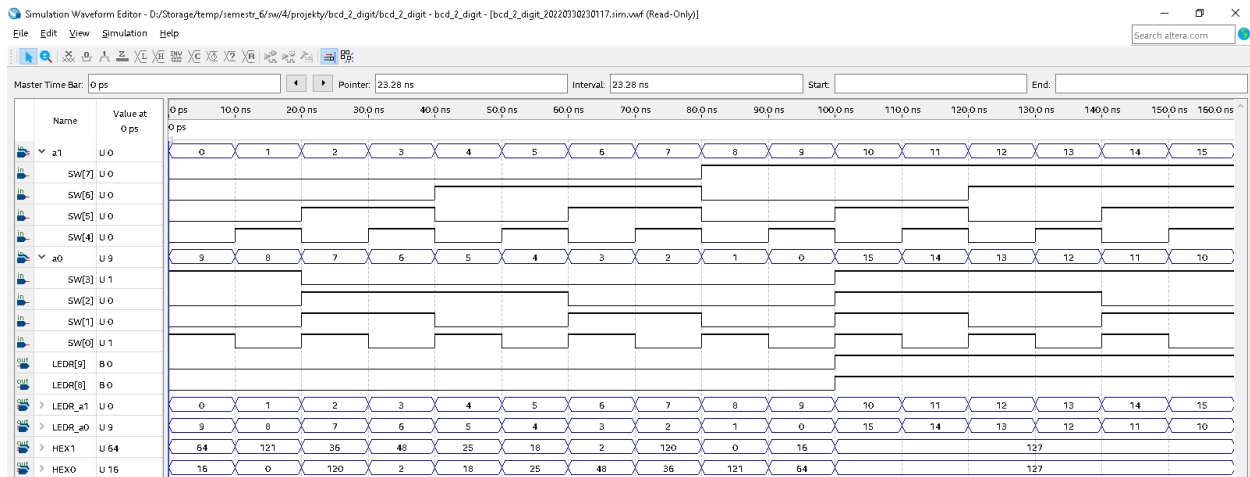
Wyniki symulacji funkcjonalnej

Przykładowo na wejście podajemy $a1a0 = 63$. Dekoder dec1 w linii `4'd6: h = 7'b0000010; // 2`

dekoduje wartość 6 i podaje na wyjściu HEX1 wartość 2. Dekoder dec0 w linii

`4'd3: h = 7'b0110000; // 48` dekoduje wartość 3 i podaje na wyjściu HEX0 wartość 48.

Gdy na wejściu a1 znajduje się wartość większa lub równa 10 nieprawidłowa w kodzie BCD, to na wyjściu LEDR[9] znajduje się 1. W przeciwnym wypadku 0. Analogicznie jest dla wejścia a0 i wyjścia LEDR[8].



Zadanie 2

Wyświetl dwucyfrową liczbę szesnastkową **B = b1b0** na wyświetlaczach **HEX1, HEX0**.

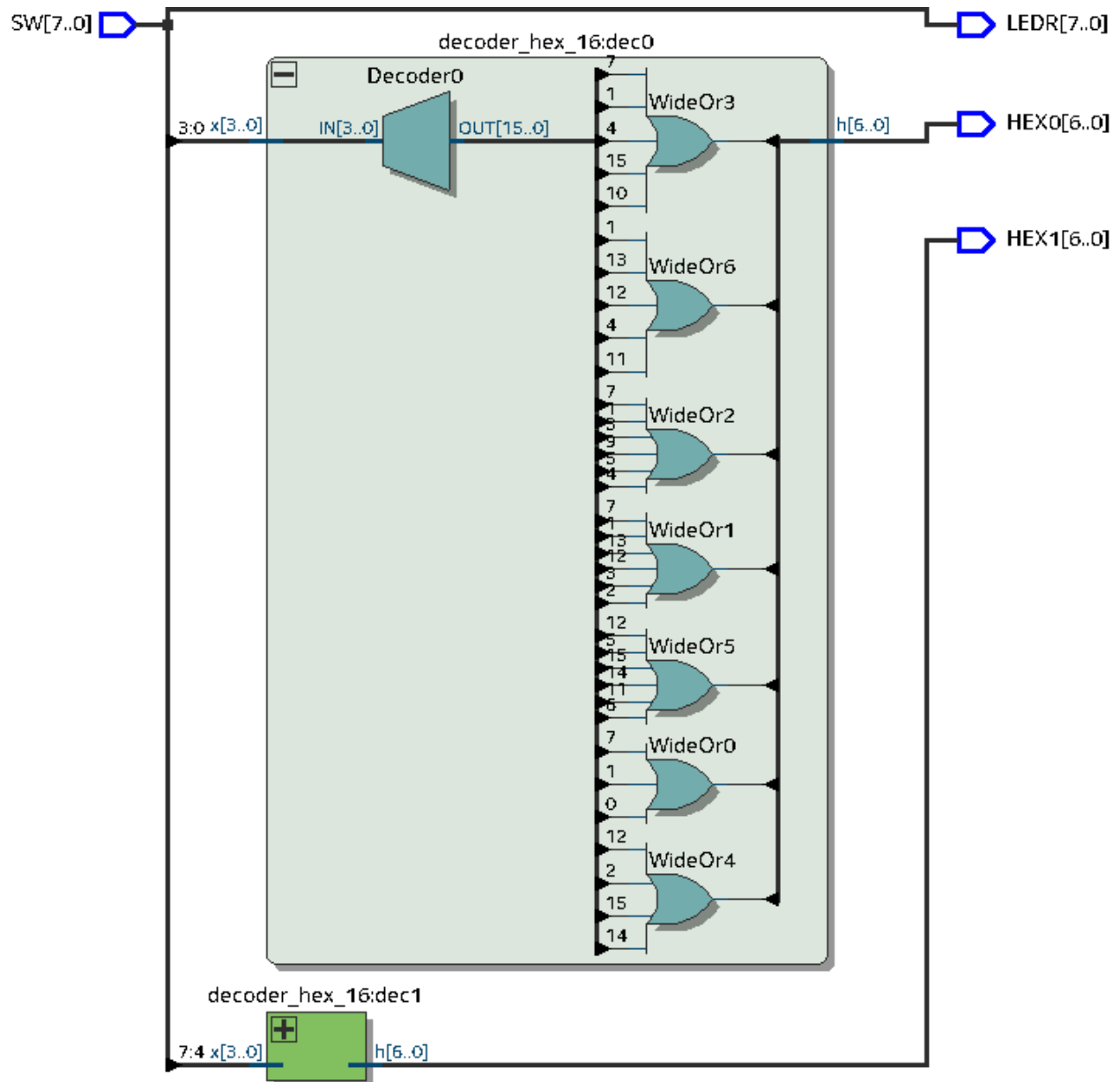
Wskazówka: opracuj projekt **decoder_hex_16 (input[3:0]x, output[0:6]h)**.

Kod w Verilog

Moduł najwyższego poziomu: hex_2_digit.

```
module hex_2_digit(  
    input [7:0] SW,  
    output [7:0] LEDR,  
    output [6:0] HEX1, HEX0);  
  
    assign LEDR[7:0] = SW[7:0];  
    decoder_hex_16 dec1(SW[7:4], HEX1[6:0]);  
    decoder_hex_16 dec0(SW[3:0], HEX0[6:0]);  
  
endmodule  
  
module decoder_hex_16(  
    input [3:0] x,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (x)  
            4'd0: h = 7'b1000000; // 64  
            4'd1: h = 7'b1111001; // 121  
            4'd2: h = 7'b0100100; // 36  
            4'd3: h = 7'b0110000; // 48  
            4'd4: h = 7'b0011001; // 25  
            4'd5: h = 7'b0010010; // 18  
            4'd6: h = 7'b0000010; // 2  
            4'd7: h = 7'b1111000; // 120  
            4'd8: h = 7'b0000000; // 0  
            4'd9: h = 7'b0010000; // 16  
            4'd10: h = 7'b0001000; // 8; A  
            4'd11: h = 7'b0000011; // 3; b  
            4'd12: h = 7'b0100111; // 39; c  
            4'd13: h = 7'b0100001; // 33; d  
            4'd14: h = 7'b0000110; // 6; E  
            4'd15: h = 7'b0001110; // 14; F  
        endcase  
  
endmodule
```

Wyniki syntezy



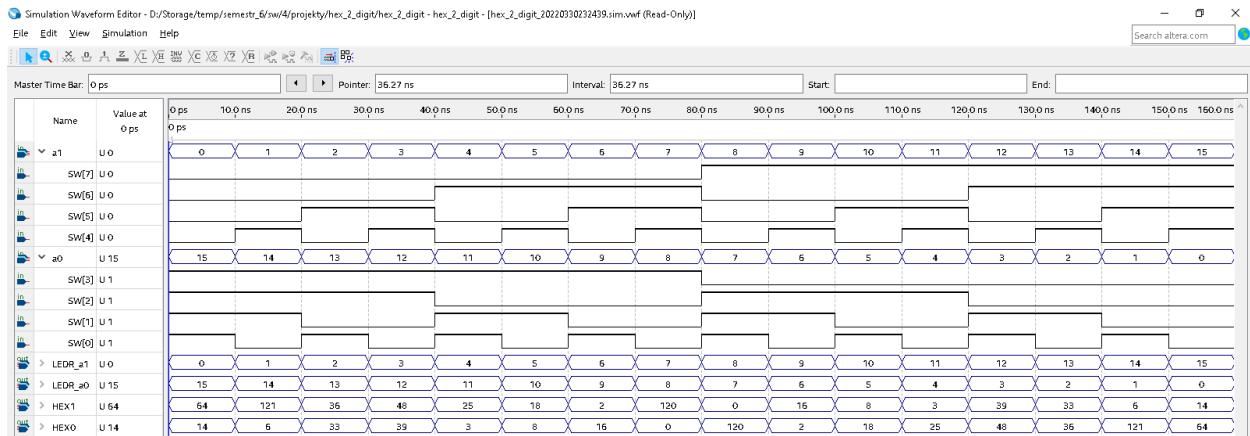
Wyniki symulacji funkcjonalnej

Przykładowo na wejście podajemy $a1|a0 = 4|11$. Dekoder dec1 w linii

`4'd4: h = 7'b0011001: // 25`

dekoduje wartość 4 i podaje na wyjściu HEX1 wartość 25. Dekoder dec0 w linii

`4'd11: h = 7'b0000011: // 3: b` dekoduje wartość 11 i podaje na wyjściu HEX0 wartość 3, która odpowiada za wyświetlenie małej litery „b”.

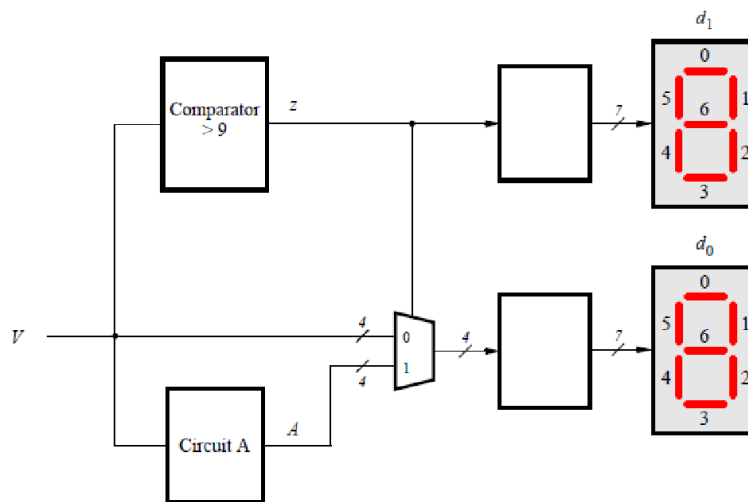


Zadanie 3

Opracuj projekt **binary_BCD_4_bits**, który konwertuje 4-bitową liczbę binarną $V = v_3v_2v_1v_0$ do postaci dwucyfrowej $D = d_1d_0$ w kodzie BCD (tab. 1, rys. 1).

Tabela 1. Wartości konwersji z kodu binarnego na dziesiętny

$v_3v_2v_1v_0$	d_1	d_0
0000	0	0
0001	0	1
0010	0	2
...	...	...
1001	0	9
1010	1	0
1011	1	1
1100	1	2
1101	1	3
1110	1	4
1111	1	5



Rys. 1. Struktura projektu układu konwersji z systemu binarnego na dziesiętny

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module binary_bcd_4_bits_on_board(
    input [3:0] SW,
    output [6:0] HEX1, HEX0);

    wire [3:0] d1, d0;
    binary_bcd_4_bits converter(SW, d1, d0);
    decoder_hex_10 dec_d1(d1, HEX1);
    decoder_hex_10 dec_d0(d0, HEX0);

endmodule
```


Dekoder z kodu binarnego na kod wyświetlacza 7-segmentowego

```
module decoder_hex_10(  
    input [3:0] binary,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (binary)  
            4'd0: h = 7'b1000000; // 64  
            4'd1: h = 7'b1111001; // 121  
            4'd2: h = 7'b0100100; // 36  
            4'd3: h = 7'b0110000; // 48  
            4'd4: h = 7'b0011001; // 25  
            4'd5: h = 7'b0010010; // 18  
            4'd6: h = 7'b0000010; // 2  
            4'd7: h = 7'b1111000; // 120  
            4'd8: h = 7'b0000000; // 0  
            4'd9: h = 7'b0010000; // 16  
            default: h = 7'b1111111; // 127  
        endcase  
endmodule
```

Konwerter 4-bitowej liczby binarnej z przedziału <0, 15> do postaci dwucyfrowej w kodzie BCD

```
module binary_bcd_4_bits(  
    input [3:0] v,  
    output [3:0] d1, d0);  
  
    wire v_greater_than_9 = (v > 4'd9); // Comparator > 9  
    wire [3:0] v_minus_10 = v - 4'd10; // Circuit A  
    wire [3:0] lower_bcd_digit;  
    mux_2_1_4_bits mux(v, v_minus_10, v_greater_than_9, lower_bcd_digit);  
    assign d1 = {3'b000, v_greater_than_9};  
    assign d0 = lower_bcd_digit;  
endmodule
```

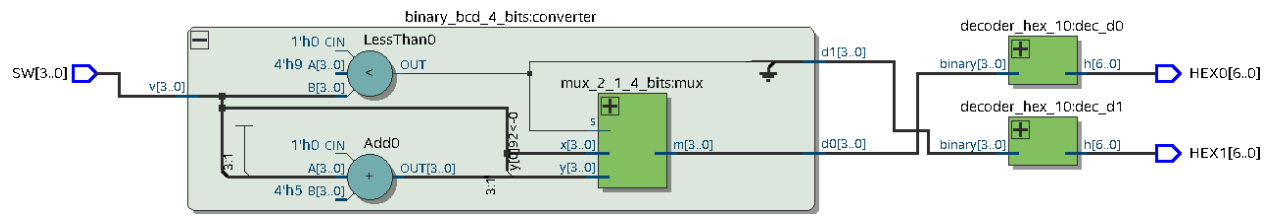
Multiplexer 2:1 4-bitowy

```
module mux_2_1_4_bits(  
    input [3:0] x, y,  
    input s,  
    output [3:0] m);  
  
    mux_2_1_1_bit mux0(x[0], y[0], s, m[0]);  
    mux_2_1_1_bit mux1(x[1], y[1], s, m[1]);  
    mux_2_1_1_bit mux2(x[2], y[2], s, m[2]);  
    mux_2_1_1_bit mux3(x[3], y[3], s, m[3]);  
endmodule
```

Multiplexer 2:1 1-bitowy

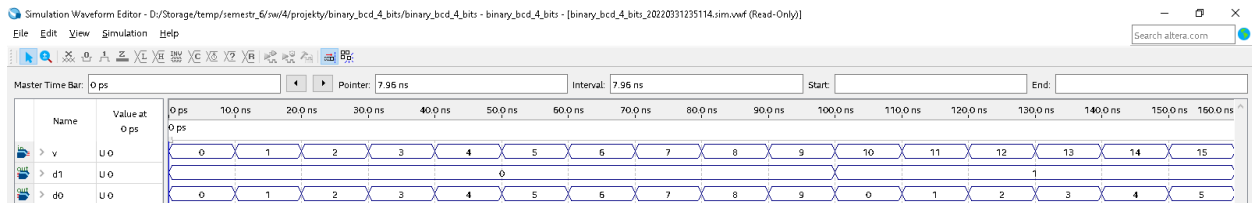
```
module mux_2_1_1_bit(  
    input x, y, s,  
    output m);  
  
    assign m = (~s & x) | (s & y);  
endmodule
```

Wyniki syntezy



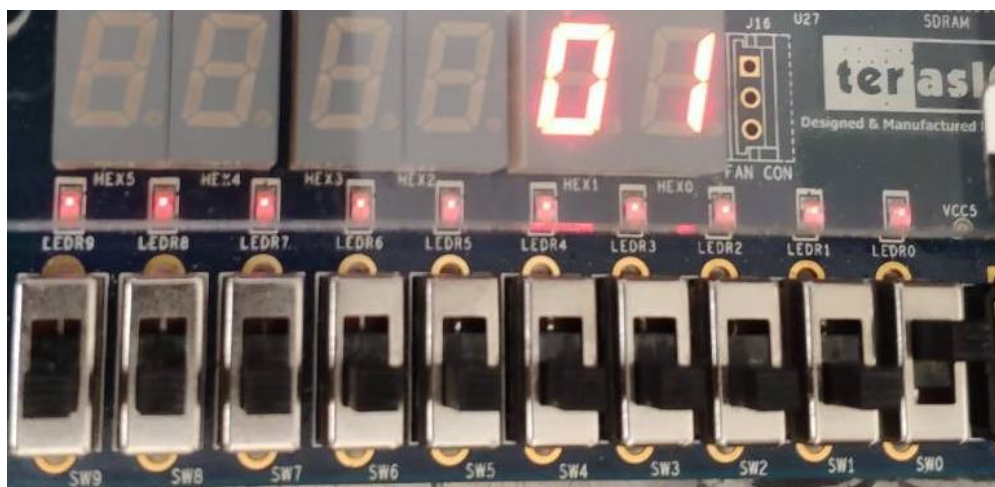
Wyniki symulacji funkcjonalnej

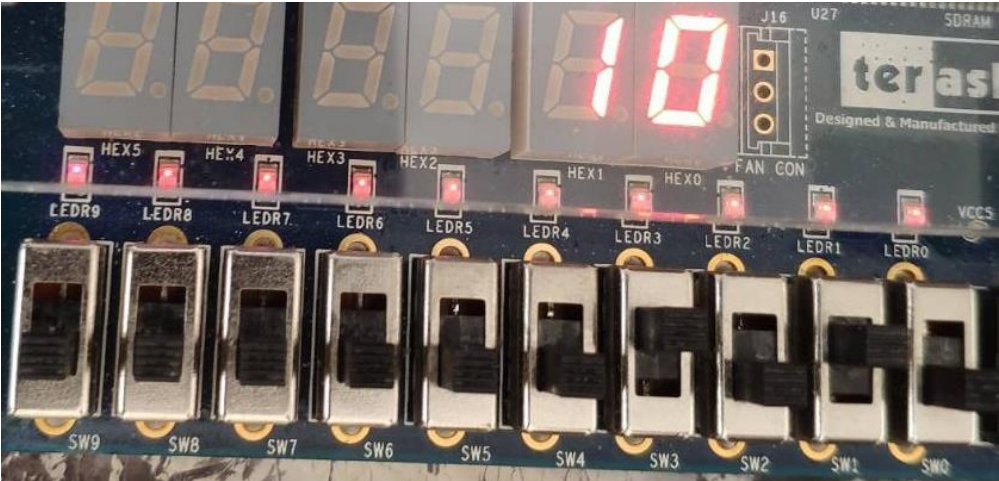
Na wyjściu d1 znajduje się cyfra dziesiątek liczby v, a na wyjściu d0 jej cyfra jedności.



Działanie na płycie







Zadanie 4

Opracuj moduł `adder_BCD_2_digits`, który dodaje dwie cyfry X i Y w kodzie BCD, i przeniesienie wejściowe cin , wynik jest reprezentowany jako dwucyfrowa suma $S = s1s0$ także w kodzie BCD.

Wskazówka: użyj układu (zamiast Circuit A), który generuje wartość najniższej cyfry kodu BCD dla sumy wartości $S \geq 10$.

Przypisania pinów:

Wyprowadzenia	Piny
X	SW[7:4], HEX5
Y	SW[3:0], HEX3
cin	SW[8]
S0	HEX0
S1	HEX1
error	LEDR[9]
SW[7:0]	LEDR[7:0]

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module adder_bcd_2_digits_on_board(  
    input [8:0] SW,  
    output [6:0] HEX5, HEX3, HEX1, HEX0,  
    output [9:0] LEDR);  
  
    assign LEDR[8:0] = SW[8:0];  
    decoder_hex_10 dec_x(SW[7:4], HEX5);  
    decoder_hex_10 dec_y(SW[3:0], HEX3);  
    wire [3:0] s1, s0;  
    adder_bcd_2_digits adder(SW[7:4], SW[3:0], SW[8], s1, s0, LEDR[9]);  
    decoder_hex_10 dec_s1(s1, HEX1);  
    decoder_hex_10 dec_s0(s0, HEX0);  
  
endmodule
```

Dekoder z kodu binarnego na kod wyświetlacza 7-segmentowego

```
module decoder_hex_10(  
    input [3:0] binary,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (binary)  
            4'd0: h = 7'b1000000; // 64  
            4'd1: h = 7'b1111001; // 121  
            4'd2: h = 7'b0100100; // 36  
            4'd3: h = 7'b0110000; // 48  
            4'd4: h = 7'b0011001; // 25  
            4'd5: h = 7'b0010010; // 18  
            4'd6: h = 7'b0000010; // 2  
            4'd7: h = 7'b1111000; // 120  
            4'd8: h = 7'b0000000; // 0  
            4'd9: h = 7'b0010000; // 16  
            default: h = 7'b1111111; // 127  
        endcase  
  
endmodule
```

Sumator w kodzie BCD

```
module adder_bcd_2_digits(  
    input [3:0] x, y,  
    input cin,  
    output [3:0] s1, s0,  
    output error);  
  
    assign error = ((x > 4'd9) | (y > 4'd9)); // syntezuje 2 4-bitowe komparatory  
    wire [4:0] binary_sum = {1'b0, x} + {1'b0, y} + {3'b000, cin};  
    binary_bcd_5_bits conv(binary_sum, s1, s0);  
  
endmodule
```

Konwerter 5-bitowej liczby binarnej z przedziału <0, 19> do postaci dwucyfrowej w kodzie BCD

```
module binary_bcd_5_bits(  
    input [4:0] v,  
    output [3:0] d1, d0);  
  
    wire v_greater_than_9 = (v > 5'd9); // Comparator > 9  
    /* odrzucamy najwyższy bit różnicy v - 10, bo przed odjęciem 10 suma może być równa  
    max 19 (mieści się na 5 bitach), a po odjęciu max 9 (mieści się na 4 bitach) */  
    wire [3:0] v_minus_10 = v - 5'd10; // Circuit A  
    wire [3:0] lower_bcd_digit;  
    mux_2_1_4_bits mux(v, v_minus_10, v_greater_than_9, lower_bcd_digit);  
  
endmodule
```

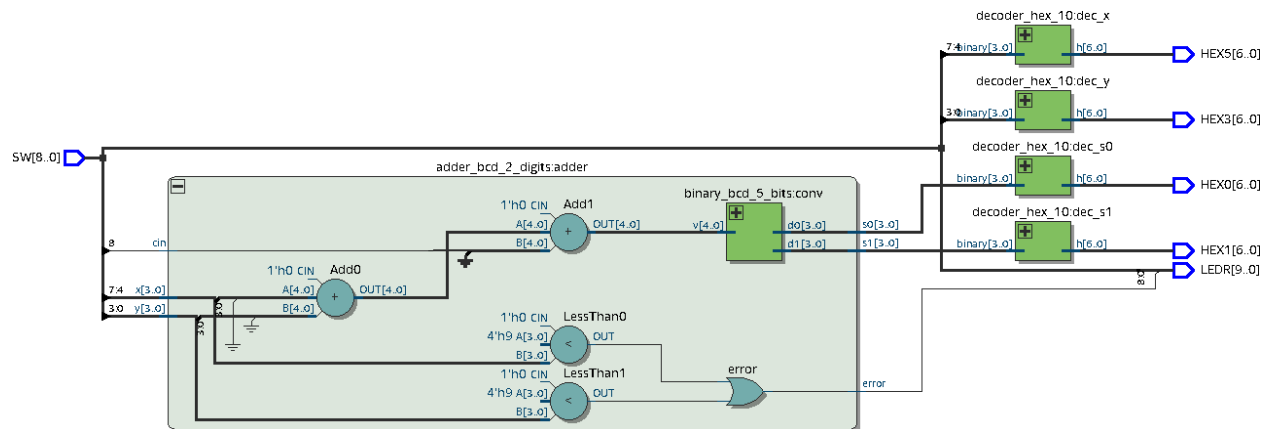
Multiplexer 2:1 4-bitowy

```
module mux_2_1_4_bits(  
    input [3:0] x, y,  
    input s,  
    output [3:0] m);  
  
    mux_2_1_1_bit mux0(x[0], y[0], s, m[0]);  
    mux_2_1_1_bit mux1(x[1], y[1], s, m[1]);  
    mux_2_1_1_bit mux2(x[2], y[2], s, m[2]);  
    mux_2_1_1_bit mux3(x[3], y[3], s, m[3]);  
  
endmodule
```

Multiplexer 2:1 1-bitowy

```
module mux_2_1_1_bit(  
    input x, y, s,  
    output m);  
  
    assign m = (~s & x) | (s & y);  
  
endmodule
```

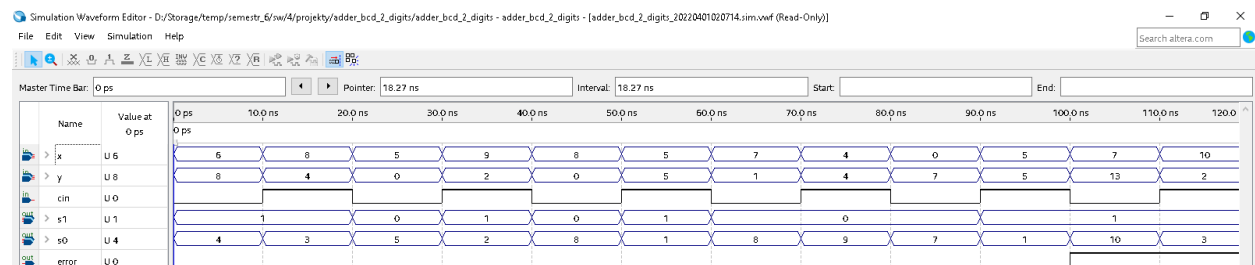
Wyniki syntezy



Wyniki symulacji funkcjonalnej

Na wyjściu s1 znajduje się w kodzie BCD cyfra dziesiątek sumy $x + y + cin$, a na wyjściu s0 jej cyfra jednostki.

Gdy na wejściu x lub y znajduje się wartość większa lub równa 10 nieprawidłowa w kodzie BCD, to na wyjściu error znajduje się 1.



Działanie na płycie

$$x (SW[7:4]) = 0100_2 = 4_{10}$$

$$y (SW[3:0]) = 0101_2 = 5_{10}$$

$$cin (SW[8]) = 0$$

X i y są mniejsze od 10, więc nie ma błędu - LEDR[9] = 0.

Suma $4 + 5 + 0 = 9$.



$$x (SW[7:4]) = 0100_2 = 4_{10}$$

$$y (SW[3:0]) = 0101_2 = 5_{10}$$

$$\text{cin} (SW[8]) = 1$$

X i y są mniejsze od 10, więc nie ma błędu - LEDR[9] = 0.

$$\text{Suma } 4 + 5 + 1 = 10.$$



$$x (SW[7:4]) = 1100_2 = 12_{10}$$

$$y (SW[3:0]) = 1010_2 = 10_{10}$$

$$\text{cin} (SW[8]) = 0$$

X i y są większe od 9, więc jest błąd - LEDR[9] = 1 i wartości wyświetlaczy HEX5, HEX3, HEX1 i HEX0 nie mają sensu.



Zadanie 5

Opracuj projekt adder_BCD_2_digits_b, który rozwiązuje poprzedni problem algorytmicznie zgodnie z następującym pseudokodem:

```
1   $T_0 = A + B + c_0$ 
2  if ( $T_0 > 9$ ) then
3       $Z_0 = 10$ ;
4       $c_1 = 1$ ;
5  else
6       $Z_0 = 0$ ;
7       $c_1 = 0$ ;
8  end if
9   $S_0 = T_0 - Z_0$ 
10  $S_1 = c_1$ 
```

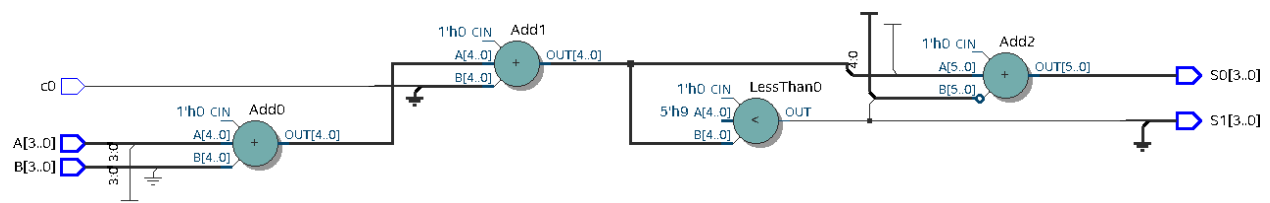
Kod w Verilog

Moduł najwyższego poziomu - sumator w kodzie BCD

```
module adder_bcd_2_digits_b(
    input [3:0] A, B,
    input c0,
    output [3:0] s1, s0);

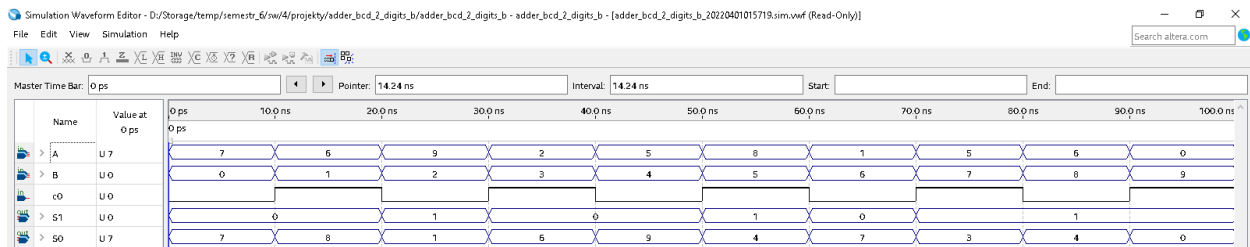
    reg c1;
    reg [3:0] z0;
    // suma A + B + c0 może być równa maksymalnie 19, a to zmieści się dopiero na 5 bitach
    wire [4:0] T0 = A + B + c0;
    always @ (*) begin
        if (T0 > 5'd9) begin
            z0 = 4'd10;
            c1 = 1'b1;
        end
        else begin
            z0 = 4'd0;
            c1 = 1'b0;
        end
    end
    // s0 jest 4-bitowe, więc najwyższy piąty bit (zawsze równy 0) różnicy T0 - z0 zostanie odrzucony
    assign s0 = T0 - {1'b0, z0};
    assign s1 = {3'b000, c1};
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej

Na wyjściu S1 znajduje się w kodzie BCD cyfra dziesiątek sumy $A + B + c0$, a na wyjściu S0 jej cyfra jedności.



Wnioski

Wszystkie zadania zostały zrealizowane.